Isolating Untrusted Tasks on Constrained Embedded Systems: A Comparative Study of ARM Cortex-M MPU and RISC-V PMP, with Real-Hardware Validation

Amadou Tidiane Anne*
Master Logiciels et Systèmes Embarqués, Université de Bretagne Occidentale (UBO)
Brest, France
amadou.tidiane.anne@proton.me

Abstract

Isolating an untrusted task within a single microcontroller core – without an MMU, without full hardware virtualization – depends on two competing families of mechanisms depending on the target architecture: the *Memory Protection Unit* (MPU) on ARM Cortex-M, and *Physical Memory Protection* (PMP) on RISC-V. Both mechanisms pursue the same goal – bounding what unprivileged code may read, write, and execute – but rest on structurally different models: fixed-priority, explicitly overlapping regions on one side; ranges addressed through stacked registers (TOR/NAPOT) on the other. This work implements the same threat scenario – an unprivileged task legitimately confined to its own memory that deliberately attempts an out-of-bounds access – on both architectures: on ARM Cortex-M4 (Nucleo-F411RE) via FreeRTOS-MPU, validated on real hardware; and on RISC-V (rv64imac) bare-metal under QEMU. We report not only the expected outcome (the illegal access is intercepted before it lands, on both platforms, verified byte-exact against the linked binary's own symbol table) but, more importantly, the process that got us there: moving to real hardware surfaced, on the ARM target, a chain of eleven real bugs – in application firmware, in the FreeRTOS-MPU port's interaction with that firmware, and in the linker script – none of which was visible from compilation or code inspection alone. That chain is itself a methodological contribution: it demonstrates concretely the gap between "compiles and looks correct" and "actually works," in a domain where that gap has a direct security cost.

Keywords

embedded security, memory protection unit, physical memory protection, RISC-V, ARM Cortex-M, FreeRTOS, task isolation, real-hardware validation

1 Introduction and Motivation

Constrained embedded systems (MMU-less microcontrollers) increasingly run heterogeneous workloads on a single core: proprietary application code, third-party libraries, and sometimes partially untrusted code (a plugin, a third-party module, a task sandboxed as defense in depth). Without a full memory management unit, isolation between these workloads can only rest on the *physical* memory protection mechanisms the core itself offers: the MPU on ARM Cortex-M, the PMP on RISC-V. These two mechanisms are functionally comparable – a small number of hardware regions, granularity constrained by alignment, an explicit privilege-escalation instruction (`SVC` on ARM, `ECALL` on RISC-V) – yet structurally distinct.

The research question is not "does isolation work on both architectures" – by construction, yes, if the hardware is configured correctly. The question is twofold:

(1) How does the *same* threat model translate across two structurally different region mechanisms, and at what configuration-complexity cost?
(2) What does moving from paper (or simulation) to real hardware reveal about the reliability of what the literature, and existing software ports, present as "ready-to-use" isolation?

This work answers both questions empirically, from two complete implementations: `freertos-stm32` (ARM Cortex-M4, real hardware) and `riscv-pmp-isolation` (RISC-V rv64imac, QEMU). Both are publicly available in the same repository as this paper.[1] It follows the same empirical, real-system-first approach as our earlier work on lightweight anomaly detection for constrained embedded Linux [2], which relied on native kernel counters rather than heavyweight instrumentation; here, the same emphasis on validating against a real, deployable system – rather than a model of one – is applied to memory isolation instead of intrusion detection.

2 Threat Model

The scenario is identical on both platforms, deliberately minimal to isolate the architectural variable:

- A "trusted" task (the RTOS kernel, or M-mode code) configures the memory protection unit to grant an unprivileged task *exclusive* access to its own stack and a small scratch working area.
- The unprivileged task runs a legitimate loop (bounded writes into its own scratch area), then, on trigger, deliberately attempts a write *outside every region granted to it* – a global variable, `g_secure_secret`, physically adjacent to its scratch region but never included in its rights.
- Success is not merely "the task was blocked" (guaranteed if the hardware is configured at all): it is that **the illegitimate write is intercepted before it reaches its target**, that the fault handler identifies the faulting address precisely, and that this address matches – to the word or byte – the protected variable, verified by cross-referencing the fault report against the linked binary's own symbol table.

That last point – cross-checking against ELF symbols rather than simply observing a fault message – is what distinguishes a genuinely instrumented demonstration from an illustrative one.

*ORCID: 0009-0003-0178-8864

[1] https://github.com/AmadouAnne/embedded-security

3 ARM Cortex-M4 Implementation (P1)

3.1 Platform and Software Architecture

Target: STM32F411RE (Cortex-M4 [3], 512 KB Flash, 128 KB RAM) on a Nucleo-F411RE board, probed live via ST-Link V2.1 (SWD, OpenOCD, GDB). RTOS: FreeRTOS V10.6.2, `ARM_CM4_MPU` port [1], `configUSE_MPU_WRAPPERS_V1 = 1`.

Four tasks coexist: three privileged system tasks (`LED` – heartbeat and hardware watchdog (IWDG) refresh; `UART` – HMAC-SHA256 challenge-response authentication and periodic reporting; `Sensor` – internal temperature ADC reading) and one `Untrusted` task, created via `xTaskCreateRestricted()` without the privilege bit, granted only its own stack and a 32-byte `scratch` region. On the `VIOLATE` command received over UART, `Untrusted` deliberately writes to `g_secure_secret`, placed by construction outside both of its regions.

The linker script (`STM32F411RETx_FLASH.ld`) explicitly reserves two "privileged-only" regions – 64 KB of Flash for kernel code and system-call entry points, 32 KB of RAM for FreeRTOS's internal structures and heap – sized as powers of two and aligned to their own size, a requirement of the Cortex-M4's region-register encoding (`RBAR`/`RASR`).

3.2 Result and Validation Method

The base system was validated on real hardware, not through a single run but through several continuous passes of 15 to 60 seconds with no reset and no fault: the scheduler runs, the hardware watchdog (IWDG) is correctly refreshed every cycle, the periodic UART report prints on schedule, and the full measurement chain (ADC → conversion → floating-point `printf` display) yields a plausible, stable temperature (26–28 °C ambient) for the whole test. Each of these four behaviors was, at some point in this work, broken by one of the bugs listed in Section 4; observing all four together and stable is itself evidence that the fixes addressed the root cause rather than papering over a symptom.

The MPU fault demonstration itself (`VIOLATE` command) now **succeeds** on real hardware. Getting there required clearing two further obstacles once the `Untrusted` task was actually scheduled (bug #10, Section 4): first, the hypothesis of stack undersizing, explicitly ruled out by test (128, 256, then 512 words, identical failure in all three cases) – a sign this was never a real overflow; then, by reading the task's stack contents live via GDB at the moment of failure, the actual cause (bug #11, Section 4): a collision between this project's own stack canary and FreeRTOS's own native overflow check, both coincidentally inspecting the exact same memory word. With that collision resolved, the `VIOLATE` command triggers a genuine MemManage fault, captured below:

```
[SECURITY] MemManage fault (unauthorized memory access)
     trapped by MPU/fault unit
  CFSR=0x00000082 MMFAR=0x20008000 BFAR=0x20008000
  -> illegal access blocked in hardware, resetting for
       isolation
[LED] tick=0
[LED] tick=501
=== Report @1021 ms === Tasks:6 Heap:12008 Temp:26.8C
```

`MMFAR` matches `g_secure_secret`'s address exactly (`arm-none-eabi-nm`), confirming the fault was raised on that specific illegal write rather than some unrelated corruption. The board resets once, by design (`report_and_reset()` in `fault_handlers.c`), then immediately resumes stable operation – behavior identical, byte-for-byte and on real silicon, to what was obtained in simulation on the RISC-V side (Sections 5.2–5.3).

4 What Real Hardware Revealed: Eleven Bugs, One Methodology

The project compiled without error and its architecture followed the documented conventions of the FreeRTOS-MPU port before it was ever flashed to real hardware. Flashing it onto the Nucleo-F411RE immediately revealed a chain of real bugs, each masking the next, discovered through direct instrumentation (GDB over the ST-Link's SWD interface, live register and memory reads, cross-referenced against the actual FreeRTOS-MPU port source and the linker script) rather than by re-reading code:

(1) **Fault-handler deadlock.** `HAL_UART_Transmit()` measures its timeout via `HAL_GetTick()`, which depends on the SysTick interrupt – but a hardware fault handler runs at a priority that blocks SysTick. The 200 ms timeout could therefore never expire: the handler meant to report the fault and reset hung forever instead. Fixed with a raw register-polling UART write with no clock dependency.

(2) **UART peripheral never actually initialized.** USART2's clock and the PA2/PA3 alternate-function mux to AF7 were never enabled – the peripheral was configured while still plain GPIO. Nothing was ever physically transmitted.

(3) **System tasks silently unprivileged.** Created via `xTaskCreate(..., priority, ...)` without `portPRIVILEGE_BIT`, which FreeRTOS-MPU requires explicitly, each of these three ordinary tasks caused an MPU fault storm on its very first context switch.

(4) **Canary planted before the stack fill that wipes it.** The stack-overflow sentinel was written *before* `xTaskCreateRestricted()`, whose own initialization fills the entire stack buffer with FreeRTOS's `0xA5` debug pattern – immediately erasing the canary and causing a permanent false-positive reset loop.

(5) **Watchdog/task-period mismatch.** The `Sensor` task's 2000 ms period exceeded the hardware watchdog's (IWDG) ≈1.6 s timeout, so the "all tasks alive" kick condition could never be satisfied within a single window – guaranteeing periodic resets regardless of any real hang.

(6) **ADC1 clock never enabled** – same bug class as #2, for the temperature sensor.

(7) **`xQueueOverwrite()` used on a length-8 queue** instead of the length-1 this function requires (an invariant enforced by `configASSERT()`): the system froze completely, interrupts disabled, on every run – externally indistinguishable from another watchdog reset loop.

(8) **`configUSE_TICK_HOOK` was 0.** FreeRTOS owns `SysTick_Handler` entirely, so nothing ever called the STM32 HAL's `HAL_IncTick()`, and every `HAL_GetTick()`-based timeout blocked forever whenever the expected hardware condition was not immediately true.

(9) **Linker script's privileged-region padding didn't reserve real space.** A `. = start + SIZE;` assignment placed *after* a section's closing brace only moves the symbolic location counter – it does not reserve that space for the placement of the next section. `.data` (and every initialized global inside it, including the HAL's own `uwTickFreq`) ended up physically inside the "privileged-only, zero-filled-at-boot" RAM carve-out, so `Reset_Handler`'s own zero-fill of that region silently erased every initialized global right after the `.data` copy loop had just set it correctly. Fixed by turning the padding into its own region-tagged section, which does correctly advance the linker's allocation cursor.

(10) **FreeRTOS-MPU's unprivileged system-call gateway placed behind the wall it exists to cross.** `.freertos_system_calls` (the `MPU_xxx` wrapper functions containing the `svc` instruction an unprivileged caller must reach to raise its own privilege) sat inside the same "privileged-execute-only" Flash carve-out as the real kernel implementation. Self-defeating by construction: unprivileged code cannot fetch the wrapper's first instruction to reach the `svc` that would let it in. Confirmed via a genuine instruction-access fault (`IACCVIOL`) the moment the `Untrusted` task, once actually scheduled, called `vTaskDelay()`. Fixed by moving the section after the privileged carve-out, where it is covered by the general "unprivileged R+X across all of Flash" region instead.

(11) **This project's own stack canary collided with FreeRTOS's native overflow check.** With bug #10 fixed and `Untrusted` finally able to run, `vApplicationStackOverflowHook()` fired on its very first context switch, for any reason at all – a silent `NVIC_SystemReset()` with no fault message. Growing the stack (128 → 256 → 512 words) changed nothing, already a sign this was not a real overflow. GDB confirmed it directly: `pxCurrentTCB->pxStack` correctly pointed at the task's stack, and words 1–3 still held FreeRTOS's own `0xa5a5a5a5` fill pattern – but word 0 held `0xDEADBEEF`, this project's own canary (bug #4), planted at the *exact same word* that `stack_macros.h`'s native `configCHECK_FOR_STACK_OVERFLOW == 2` check inspects on every context switch. Two independently correct integrity mechanisms, each blind to the other, sharing one word of memory. Fixed by moving the canary to word index 4.

None of these eleven bugs became observable until the previous one was fixed – a cascading structure typical of real hardware debugging, where one symptom systematically masks another. None would have been caught by code review, compilation, or a simulation that does not faithfully model the hardware watchdog, peripheral clocks, and the exact semantics of linker-script placement.

5 RISC-V Implementation (P7)

5.1 Platform and the PMP Model

Target: RISC-V rv64imac (the `_zicsr_zifencei` extensions must be requested explicitly on current binutils toolchains, since `zicsr` is no longer implied by the base ISA), simulated under QEMU [4] (`virt` machine), bare-metal, no RTOS. The isolation mechanism, Physical Memory Protection, differs structurally from the ARM MPU: a set of M-mode-only control and status registers (CSRs) – `pmpaddr0-15`, `pmpcfg0-3` – encode up to 16 ranges, each addressed in TOR (Top-Of-Range) mode, where entry i covers [pmpaddr($i-1$), pmpaddr(i)) [5]. Unlike the Cortex-M4 MPU (explicit base+size regions, priority by increasing region number on overlap), TOR mode builds contiguous ranges by accumulating successive addresses – a non-trivial difference in mental model when porting the same region scheme.

Another structural difference: M-mode is, by default, **exempt** from all PMP enforcement (unless an entry is explicitly locked), whereas U-mode fails closed – any address matching no PMP entry is denied by default. The memory map (RAM at `0x80000000`, NS16550A UART at `0x10000000`, `sifive_test` finisher device at `0x100000`) was confirmed empirically from QEMU's own generated device tree rather than assumed from documentation.

5.2 Result – Phase 2: Single-Task Isolation

Four PMP entries configure two effective ranges granted to a U-mode task: read+execute on its own code, read+write on its own scratch area and stack. The task legitimately touches its scratch area, then deliberately writes to `g_secure_secret`, outside every granted range. Real, captured output:

```
=== TRAP CAUGHT (M-mode) ===
Store/AMO access fault (PMP denied write)
mcause: 0x0000000000000007
mepc : 0x0000000080000074
mtval : 0x0000000080000660
```

Cross-checked against the linked ELF's own symbol table: `mtval` matches `g_secure_secret`'s address exactly, and that variable sits at the very next byte after the scratch region's granted end – the boundary is enforced precisely, not with slack. `mepc` falls inside the code granted to the unprivileged task, confirming the fault was raised from code genuinely executing in U-mode.

5.3 Result – Phase 3: Multi-Task Scheduler and Sibling-Task Isolation

Phase 2 answers "can unprivileged code reach a kernel secret." It says nothing about the **RTOS-integration cost** that a first version of this study left open on the RISC-V side. Phase 3 closes that gap: a minimal M-mode round-robin scheduler preempts two structurally identical U-mode tasks via a real hardware interrupt (the `virt` machine's CLINT timer [6], confirmed at 10 MHz from the machine's own device tree) and **reconfigures the PMP entries on every context switch** – the exact structural counterpart, on the RISC-V/PMP side, to `xTaskCreateRestricted()`'s per-task `MemoryRegions` being rearmed on every FreeRTOS context switch (Section 3).

TaskA runs legitimately for several scheduling quanta (both counters advance genuinely interleaved, not merely alternating, proof that preemption is real), then deliberately writes into TaskB's own counter – memory neither task shares, isolated only by the PMP regions the scheduler swaps in and out on every switch. Real, captured output:

```
[sched] tick -- TaskA=counter: 0x0000000000000004
[sched] TaskB=counter: 0x0000000000000005
```

```
=== TRAP: PMP violation ===
Task: TaskA
Store/AMO access fault (PMP denied write)
mepc : 0x0000000080000078
mtval: 0x0000000080000d00
[sched] tick -- TaskA=counter: 0x0000000000000005
[sched] TaskB=counter: 0x0000000000000005
[sched] tick -- TaskA=counter: 0x0000000000000005
[sched] TaskB=counter: 0x0000000000000006
```

`mtval` matches the exact address of TaskB's counter, the very first byte of its own data region – one byte past the end of the region granted to TaskA. The offending task is immediately dropped from scheduling (its counter is frozen in every subsequent sample), while TaskB, granted nothing by TaskA's PMP configuration and never touched by it, keeps advancing without interruption. The violation was **contained at its source**, not merely detected – the difference between a protection mechanism and an incident log.

This phase itself produced a transferable methodological observation, in the spirit of Section 4: a first bug – forgetting to configure the PMP for the first task before the very first return to user mode, since PMP resets with every entry disabled and U-mode then fails closed on absolutely everything, including its own first instruction – was found and fixed quickly. A second symptom, however, was actively mistaken for a context-switch correctness bug (wrong register restored, stale program counter) for a non-trivial amount of GDB-based investigation, when it was in fact a speed artifact: the tasks' busy-wait loop, sized for real silicon, took several real seconds per full pass under QEMU's software emulation – far longer than any observation window used to diagnose it. A timing-sensitive symptom can be visually indistinguishable from a logic error; a counter appearing frozen does not by itself indict the algorithm that advances it.

5.4 Explicit Limitation: Simulation-Only Validation

Unlike P1, this result is **not yet validated on real silicon**: no RISC-V board was available at the time of this work (explicitly checked via `lsusb`: only this machine's unrelated ST-Link, for the ARM side of this comparison, was detected). Phase 4 (hardware validation, e.g. on an ESP32-C3) remains open. This work fully owns that limitation rather than hiding it – precisely because P1 revealed a chain of eleven bugs invisible outside real hardware (two of them found after this study's first draft, see Section 4), P7's QEMU result, however clean it now is for Phases 2 and 3, must be presented as provisional until it has faced the same trial.

6 Comparative Analysis

This asymmetry is deliberate, not an accidental imbalance: P1 was pushed to completion on the hardware-reliability axis (eleven bugs found because real hardware was used as the judge, all the way to a complete, reproducible fault demonstration), while P7 was pushed to completion on the functional-coverage axis (peer isolation, not merely task-versus-kernel). Each implementation reached its own axis fully; neither yet covers both – P1 has not been extended to sibling-task isolation, P7 has not yet faced real silicon.

Table 1: What was actually demonstrated on each platform.

	ARM Cortex-M4 (P1)	RISC-V rv64imac (P7)
Task/kernel-secret isolation	demonstrated, byte-exact, on real hardware	demonstrated, byte-exact, in simulation
Sibling-task isolation	not tested in this work	demonstrated, fault contained, sibling unaffected
Real bugs found & fixed	11, on real hardware	2, in simulation
Validation	real hardware (Nucleo-F411RE)	simulated (QEMU), hardware pending

Table 2: Mechanism comparison: ARM MPU vs. RISC-V PMP.

	Cortex-M4 (MPU)	rv64imac (PMP)
Regions (tested target)	8 hardware regions	up to 16 PMP entries
Region addressing	base+size, power-of-two, aligned	TOR: successive addresses, or NAPOT
Privilege escalation	`SVC`, `vPortSVCHandler`, caller-range check	`ECALL`, software-handled in M-mode
Default outside-region behavior	configuration-dependent (`PRIVDEFENA`)	U-mode: always fails closed; M-mode: exempt unless locked
Multi-task scheduling w/ per-task reconfiguration	yes (FreeRTOS-MPU, dedicated `ARM_CM4_MPU` port)	yes (minimal M-mode scheduler, written for this work)
Sibling-task isolation verified	not tested in this work	yes (Phase 3, Sec. 5.3)
Validation	real hardware	simulated, hardware pending

The central finding is not that one mechanism is superior: both achieve the intended security objective, with comparable precision (word-exact interception, verified against binary symbols). The significant difference lies in **integration surface**. On ARM, isolation is inseparable from the full RTOS port (`ARM_CM4_MPU`, `MPU_WRAPPERS_V1`): the majority of the eleven bugs encountered (#3, 4, 9, 10, 11) come precisely from that integration – the interaction between the region scheme, the scheduler, the linker script, port conventions, and even this project's own security code – rather than from the MPU mechanism in isolation. On RISC-V, Phase 3's minimal scheduler performs substantially the same integration (per-switch region reconfiguration, coexistence with a hardware interrupt) with a much shorter bug chain (two, versus eleven on

ARM) – but this raw comparison should be read cautiously: the RISC-V scheduler here is minimal code written specifically for this work, not a widely-deployed third-party port following external conventions like `ARM_CM4_MPU/MPU_WRAPPERS_V1`, and it has not yet faced the real-hardware trial that precisely revealed most of the ARM-side bugs. The observed gap may say as much about relative implementation maturity as about the mechanisms themselves.

7 Discussion: Beyond the Result, the Method

The most generalizable contribution of this work may not be the architectural comparison itself, but the measured gap between **code judged correct at compile time, following the documented conventions of a widely-used port (FreeRTOS-MPU)**, on one hand, and **actual behavior on silicon**, on the other. The eleven bugs listed in Section 4 are not isolated beginner mistakes: they touch subtle interaction points – the exact semantics of location-counter assignment in a GNU ld linker script, the relative order between planting a canary and an RTOS task's own internal initialization, an implicit dependency of a vendor HAL on a clock mechanism the RTOS fully appropriates, or two independently correct security mechanisms that, with no explicit coordination, end up inspecting the same memory word (bug #11). Each is individually plausible in any FreeRTOS-MPU project following the same conventions; their accumulation, undetected before the first real flash, concretely illustrates why a security review of physically-isolated memory systems cannot stop at static inspection of the declared region scheme.

8 Limitations and Future Work

- RISC-V validation is currently limited to QEMU simulation; Phase 4 (real-hardware validation) remains to be done. Phase 3 (multi-task scheduler reconfiguring PMP on every context switch) is now complete (Sec. 5.3), but itself unvalidated on silicon – and, as discussed in Section 6, it is minimal code written for this work, not a widely-deployed third-party port: the integration-cost comparison between the two architectures thus remains partial until the RISC-V scheduler has faced a comparable maturity trial (real usage, real hardware, external review).
- The UART HMAC-SHA256 challenge-response authentication protocol (P1) has not yet been tested under sustained real load.
- P1's sibling-task isolation (the RISC-V-side result of Section 5.3) has not been implemented or tested; extending FreeRTOS-MPU's per-task region model to two structurally identical, mutually-isolated unprivileged tasks on the ARM side is natural future work to close the last asymmetry in Table 1.

9 Conclusion

This work poses the same security question – can an unprivileged task be confined to its own memory such that a violation is intercepted before it lands, rather than silently corrupting something else – on two structurally different physical memory-protection mechanisms, and answers it twice, with two complementary forms of rigor rather than one form repeated twice.

On ARM Cortex-M4, rigor bears on real hardware: a FreeRTOS-MPU system that compiled without error and followed the documented conventions of the `ARM_CM4_MPU` port turned out, from the very first flash onto a Nucleo-F411RE, to carry a chain of eleven real bugs – in application firmware, in its interaction with the RTOS port, and in the linker script – none visible from code review or caught by compilation. The base system emerges stabilized and verified over tens of seconds of continuous operation, and the fault demonstration itself now succeeds as well: the `VIOLATE` command triggers a genuine MemManage fault on real silicon, whose faulting address matches the protected variable exactly, byte for byte. The chain's last two bugs (#10 and #11) themselves followed the methodology described in Section 6 to completion – each found, root-caused by live memory inspection under GDB, and fixed – rather than being left as a documented failure.

On RISC-V, rigor bears on functional coverage: beyond the question "can unprivileged code reach a kernel secret" (Phase 2, answered and verified byte-exact), a minimal M-mode scheduler written for this work demonstrates the question structurally closer to real deployment – two sibling tasks isolated from one another by nothing but PMP regions reconfigured on every context switch, where the offending task is killed and contained while its sibling continues execution with no measurable interruption (Phase 3). This demonstration has, at this stage, faced neither the trial of real hardware nor that of external use – two limitations explicitly owned rather than concealed.

The most generalizable result of this work is therefore neither "MPU" nor "PMP" in isolation, but the conjunction of both experiments: a physical memory-isolation mechanism can be correctly *designed* – sound architecture, conventions respected, favorable code review – and still be wrong until it has been tested, whether by real silicon (as P1 demonstrated through eleven bugs, all the way to a complete demonstration) or by a threat scenario more demanding than the one initially targeted (as P7 demonstrated by moving from task-versus-kernel isolation to peer isolation). In a domain where memory isolation is a security mechanism, not a performance optimization, neither code review, nor simulation, nor even a first successful demonstration is sufficient to establish reliability – only sustained trial, on real hardware and against scenarios increasingly close to final deployment, can. It is that trial, more than any single result at any single point in time, that this work documents.

Data and Code Availability

Complete source code for both implementations, build instructions, and the full commit history documenting each bug's discovery and fix are publicly available at https://github.com/AmadouAnne/embedded-security, under the `freertos-stm32` (P1) and `riscv-pmp-isolation` (P7) directories. In particular:

- **P1 (ARM)** requires a Nucleo-F411RE board, an ST-Link probe, the `arm-none-eabi` toolchain, and OpenOCD. `make` builds the firmware; `openocd -f openocd/stm32f4.cfg -c "program build/freertos_hardened.bin 0x08000000 verify reset exit"` flashes it. The `VIOLATE` command, sent over the board's UART (115200 8N1) once the system is running, triggers the fault demonstration of Section 3.2.

- **P7 (RISC-V)** requires only the `riscv64-elf` toolchain and QEMU (`qemu-system-riscv64`) – no hardware. `make run` builds and boots the Phase 3 scheduler demonstration of Section 5.3 directly, with UART routed to standard I/O.

Every real captured output quoted in this paper (Sections 3.2, 5.2, and 5.3) can be reproduced exactly with the commands above, against the exact commit referenced by this paper's own repository history.

References

[1] Amazon Web Services. 2024. FreeRTOS-MPU Memory Protection Unit Support. https://www.freertos.org/Why-FreeRTOS/Demos/mpu-arm-cortex-m. Accessed 2026.

[2] Amadou Tidiane Anne. 2026. *Analyse de la pertinence des métriques système natives pour la détection d'anomalies sous Linux en environnements contraints.* Preprint hal-05486729. HAL. https://hal.science/hal-05486729v1

[3] ARM Limited. 2021. *ARMv7-M Architecture Reference Manual.* Technical Report. ARM Limited. DDI 0403E.e.

[4] Fabrice Bellard. 2005. QEMU, a Fast and Portable Dynamic Translator. In *USENIX Annual Technical Conference, FREENIX Track*. 41–46.

[5] RISC-V International. 2024. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture.* Technical Report. RISC-V International. Document Version 20240411.

[6] SiFive, Inc. 2021. SiFive E31 Core Complex Manual: CLINT. https://sifive.cdn.prismic.io/sifive/. Core-Local Interruptor specification.